

NEXUS AI

Data, Memory & Operations Document

N55—N62 • Database • Memory • Conversations • Projects • Configuration • Observability • Storage • Backup

Version: 0.1 **Status:** Data & Operations Foundation **Mode:** Local-first

Purpose: Define how NEXUS persists conversations, projects, memory, knowledge metadata, configuration, logs and recovery data on the user's device while preserving privacy, integrity and predictable lifecycle behavior.

APPLICATION → DATA SERVICES → DATABASE / MEMORY / FILE STORAGE → BACKUP / RECOVERY

1. Data Architecture Vision

NEXUS uses a local persistence architecture that separates structured application data, semantic knowledge indexes, files, runtime configuration and operational telemetry. The goal is to keep user data understandable, recoverable and under user control.

Core Principles

- Local-first persistence for core product data.
- Clear ownership and scope for every stored object.
- Project and collection boundaries enforced in application logic.
- Sensitive content excluded from logs unless explicitly required.
- Derived data can be rebuilt from authoritative sources.
- Schema and data versions are tracked for safe upgrades.
- Deletion is explicit and cascades to dependent derived data.
- Backups prioritize user-created data and configuration.
- Operational telemetry is bounded and privacy-aware.

2. N55 — Database Schema Document

The primary relational database stores application state, metadata, task state and references. Large source files and model files should not be stored as database blobs unless there is a specific reason.

Entity	Purpose	Key Relationships
users/local_profile	Single local user/profile settings	Owns projects and preferences
projects	Project/workspace metadata	Contains conversations, tasks, files and memory
conversations	Chat sessions	Belongs to project or global scope
messages	Individual chat messages	Belongs to conversation
models	Local model registry metadata	Referenced by routing/tasks
model_runs	Inference execution records	References model + task/message
knowledge_collections	RAG collection metadata	Scoped to project/user
documents	Source document metadata	Belongs to collection
document_versions	Content/version tracking	Belongs to document
chunks	Retrieval units + provenance	Belongs to document version
agent_tasks	Long-running AI tasks	Belongs to project/session
agent_steps	Plan/execution state	Belongs to agent task
tool_calls	Tool execution records	Belongs to agent step/task
permissions	Policy/configuration records	Scoped to project/tool
memory_items	Persistent memory records	Scoped to project/user
settings	Application configuration metadata	Local scope
events/logs	Operational/audit events	References task/session where useful

Database Design Rules

Use stable UUID-style identifiers, UTC timestamps internally, explicit status fields, foreign-key relationships where appropriate, indexes on common lookup fields and soft-delete/version fields where recovery or audit requirements justify them.

3. Logical Schema Groups

Group	Tables / Records	Authority
Identity & Settings	profile, settings, permissions	Database/config

Group	Tables / Records	Authority
Conversation	conversations, messages, message_metadata	Database
Projects	projects, workspace metadata, tasks	Database + filesystem
AI Runtime	models, model_runs, routing records	Database
Knowledge	collections, documents, versions, chunks, indexes	Database + vector store/files
Agents	agent_tasks, steps, tool_calls, approvals	Database
Memory	memory_items, memory_links	Database/vector index where used
Operations	events, errors, metrics summaries	Database/log files

4. N56 — Memory Storage Architecture

NEXUS memory is separated by scope so that unrelated information is not injected into prompts.

Memory Type	Scope	Storage
Conversation memory	Current/past chat	Relational message store + optional summaries
Session/task memory	Current execution	Task/step state
Project memory	One project	Relational records + semantic index where useful
Knowledge memory	Documents/evidence	RAG collections/vector index
User memory	Explicit approved preferences/facts	Local relational store + optional vector index
Agent memory	Role/task-specific reusable state	Controlled agent memory store

Memory Record

memory_id, scope_type, scope_id, content/reference, memory_type, importance, source, confidence, created_at, updated_at, expires_at where applicable, embedding/index reference where applicable, user_visible flag and deletion state.

Memory Retrieval

Current task → determine relevant memory scope → retrieve candidate memories → filter by permissions/freshness → rank → inject only the required subset into context.

5. Memory Lifecycle

Candidate → Evaluated → Active → Updated/Consolidated → Expired/Archived → Deleted.

Not every conversation statement becomes memory. NEXUS should prefer explicit user-approved memory or strongly useful project facts over automatic storage of every message.

6. N57 — Conversation Data Model

Conversations are durable interaction records that support streaming, branching/future extension and model/tool metadata.

Object	Important Fields
Conversation	id, project_id, title, mode, model_policy, created_at, updated_at, archived_at
Message	id, conversation_id, role, content/reference, status, created_at
Message metadata	model_id, task_id, routing_mode, token metrics, latency
Attachment	id, message_id, file/reference, type, size, processing state
Source reference	message_id, document/chunk reference, citation metadata
Generation	run_id, start/end, status, stop reason, error

Message Roles

user, assistant, system/application metadata, tool and tool-result are represented distinctly. Hidden internal reasoning is not stored as ordinary assistant content.

Streaming

A generation creates a run record, streams deltas to the UI, then finalizes one durable assistant message. Partial output is marked interrupted if generation is cancelled or fails.

7. N58 — Project Workspace Data Model

Projects are the primary isolation boundary for files, conversations, knowledge and agent tasks.

Object	Purpose
Project	Identity, description, workspace root, defaults and permissions.
Workspace	Filesystem metadata and root configuration.
Project file reference	Metadata/reference to a real file; file bytes remain on disk.
Task	Project-level work item or agent task.
Project memory	Approved decisions and facts.
Knowledge collection link	Associates project with RAG collections.
Project setting	Project-specific model/tool preferences.

Workspace Rule

The database stores metadata and references; the filesystem stores actual project files. This prevents the database from becoming a giant binary/code repository.

8. N59 — Configuration Management Plan

Configuration is divided into application defaults, user preferences, project configuration and secure/runtime configuration.

Configuration	Examples	Storage
Application	UI defaults, feature flags	Local config/database
User	Theme, default model, privacy preferences	Database/config
Project	Workspace path, model policy, tools	Database
Runtime	Ollama endpoint, ports, paths	Config/OS environment
Security	Permission policies, approval defaults	Protected local config/database
Secrets	API keys if optional integrations exist	OS secure storage where available; never plain logs

Configuration Rules

- Provide safe defaults.
- Validate configuration at startup and on change.
- Version configuration schema.
- Separate secrets from normal settings.
- Never require cloud credentials for the offline core.
- Expose important local paths and data controls in Settings.
- Support export/import of non-secret configuration where useful.

9. Configuration Precedence

System defaults → application config → user preferences → project settings → task/session overrides. More specific configuration may override less specific configuration only where the property is designed to be overrideable.

Security and permission policies are not freely overridden by prompts or project text.

10. N60 — Logging & Observability Plan

Observability is required to debug local AI workflows, but logs must not become an uncontrolled copy of user data.

Signal	Purpose	Retention
Application logs	Errors, startup, service health	Short/rotating
AI run telemetry	Model, latency, status, resource metrics	Configurable
RAG telemetry	Retrieval count, latency, collection IDs	Bounded
Agent events	Task/step/tool status	Task lifecycle + configured history
Security/audit	Permission and high-risk actions	Longer, user-controlled
Crash reports	Failure diagnosis	Local by default

Structured Event Fields

timestamp, event_type, severity, component, task_id/session_id where applicable, duration, status, error_code and sanitized metadata.

Privacy Rules

Do not log full prompts, document contents, secrets, credentials or full tool output by default. Debug mode may expose more information only with explicit user control and clear warnings.

11. Metrics

Metric	Purpose
App startup time	Local UX health.
Model TTFT / throughput	Inference performance.
Model error rate	Runtime reliability.
RAG latency	Knowledge performance.
Agent completion rate	Task reliability.
Tool failure rate	Execution reliability.
Database operation latency	Persistence health.
Disk usage	Storage management.
Crash count	Application stability.

12. N61 — Local Storage & Data Lifecycle Plan

NEXUS storage is divided into authoritative data, derived data, cache and logs.

Storage Class	Examples	Lifecycle
Authoritative	Projects, conversations, settings, memory	Keep until user deletes/archives
Source	Imported documents, project files	User-controlled
Derived	Embeddings, chunks/indexes, summaries	Rebuildable; tied to source/version
Cache	Temporary model/RAG/UI cache	Automatically cleanable
Logs	Operational events	Rotating retention
Backups	Snapshots/export packages	User-defined retention

Suggested Local Layout

Application Data / Database / Knowledge Index / Cache / Logs / Backups / Workspaces. Exact OS-specific paths are implementation decisions, but the user must be able to identify the locations from Settings.

13. Storage Lifecycle Rules

Create → Active → Updated → Archived → Deleted. Derived records carry source/version references. When a source is deleted, associated derived indexes and memory references are invalidated according to policy.

Cleanup

- Cache cleanup should never delete authoritative user data.
- Logs rotate automatically.
- Old document versions may be retained according to collection policy.
- Failed temporary ingestion artifacts are cleaned after job completion/failure.
- Large unused indexes can be rebuilt instead of blindly retained.

14. Disk & Resource Management

Local AI can consume significant disk space through models, embeddings, caches and documents. NEXUS should show storage usage by category and warn before critical thresholds.

Category	UI Metric
Models	Installed model size.
Knowledge	Source + index size.
Projects	Workspace size.
Database	Database size.
Logs/Cache	Current temporary/operational usage.
Backups	Snapshot/export size.

15. N62 — Backup & Recovery Plan

Backup focuses on user-created and configuration data while allowing large derived artifacts to be rebuilt when practical.

Backup Target	Priority	Recovery
Database	Critical	Restore directly.
Project files	Critical	Restore filesystem snapshot/export.
Knowledge source documents	Critical	Restore source files; rebuild indexes.
Settings/config	High	Restore/import.
Memory	High	Restore database records.
Vector indexes	Medium	Restore or rebuild from source.
Cache	Low	Recreate.
Logs	Low–Medium	Optional diagnostic archive.

Backup Modes

Manual Export — user-triggered backup. Scheduled Local Backup — periodic snapshot to a user-selected location. Full Export — database + projects + knowledge sources + configuration. Portable Project Export — one project and its required metadata.

Backup Safety

- Never overwrite the only known backup without a recovery strategy.
- Use versioned backup names.
- Validate backup completeness after creation.
- Include schema/application version metadata.
- Encrypt backups when sensitive data requires it and a secure local mechanism is available.
- Do not include secrets in portable exports unless explicitly selected.

16. Recovery Scenarios

Scenario	Recovery Strategy
Database corruption	Restore latest valid backup; verify schema.
Application update failure	Rollback application; retain data directory.
Deleted project	Restore project files + database snapshot or project export.
Knowledge index corruption	Rebuild index from source documents.
Model cache failure	Re-download/reload models through configured runtime.
Config corruption	Reset to defaults and restore user export.
Partial agent task	Resume from durable checkpoint where safe.

17. Startup Integrity Checks

At startup NEXUS should validate database connectivity, schema version, storage paths, workspace availability, knowledge index health, Ollama connectivity and critical configuration. A failure should isolate the affected subsystem rather than making the entire application unusable.

Example: corrupted RAG index → Chat still works → Knowledge Center shows repair required → user can rebuild index.

18. Database Migration Strategy

Every schema change is versioned and applied through migrations. Startup checks the current schema version before enabling dependent features.

Migration Rule	Requirement
Versioned	Every schema change has an ordered version.
Backward awareness	Application knows which schema versions it supports.
Backup	Important migrations should be preceded by a backup/checkpoint.
Validation	Post-migration integrity checks.
Rollback	Provide rollback or recovery strategy where feasible.
Data transform	Transform scripts must be deterministic and tested.

19. Data Integrity

- Use foreign-key relationships where supported.
- Use unique constraints for stable identities.
- Validate status transitions.
- Use transactions for multi-record state changes.
- Store checksums for important source files.
- Detect orphaned derived records.
- Periodically validate knowledge/index references.
- Protect active workspace paths from accidental deletion.

20. Operational State Machine

Healthy → Degraded → Recovery Required → Recovering → Healthy. Individual subsystems may have independent health states.

Subsystem	Health Checks
Database	Connection, migration state, integrity.
Ollama	Reachability, model discovery, inference test.
Knowledge	Index availability, version consistency.
Filesystem	Workspace path access.

Subsystem	Health Checks
Agent Runtime	Worker/task queue health.
Storage	Free space and write access.

21. Data Security & Privacy

NEXUS is local-first, but local does not automatically mean secure. The application should protect data directories with OS-level access controls and avoid unnecessary exposure.

- Store application data in a user-specific protected location.
- Do not print secrets into logs.
- Separate optional online integrations from offline core.
- Make network-enabled features visible.
- Provide a clear data deletion workflow.
- Protect backups and exports because they contain user data.
- Treat imported documents and project files as untrusted content.

22. Operations Dashboard

An internal diagnostics screen should show subsystem health without requiring the user to understand implementation details.

Panel	Information
Runtime	Ollama status, active model, resource state.
Database	Health, schema version, size.
Knowledge	Collections, index status, failed jobs.
Agents	Active tasks, queue, failures.
Storage	Usage by category.
Logs	Recent errors/events with filters.
Backup	Last backup, status, location and validation.

23. Data Export & Portability

NEXUS should support export of conversations, projects, memory and selected knowledge metadata in documented formats. A full portable export should include an application/schema version manifest so future versions can migrate it safely.

Export Rule

Export should distinguish source data from derived data. A portable project can include source files and database metadata while allowing embeddings/indexes to be rebuilt.

24. Data Lifecycle Acceptance Gate

- Core application data persists across restart.
- Conversation history is durable and correctly ordered.
- Project data is isolated by project scope.
- Memory can be stored, retrieved and deleted.
- Configuration changes persist and validate.
- Logs rotate without deleting authoritative data.
- Derived RAG data can be rebuilt from sources.
- Backup can be created and successfully validated.

- Recovery can restore a representative project.
- Storage usage is visible to the user.

25. Implementation Order for N55–N62

1. Database foundation → 2. migrations → 3. project/conversation schema → 4. configuration → 5. agent/task persistence → 6. memory storage → 7. RAG metadata/index references → 8. logging/events → 9. storage lifecycle → 10. backup/export → 11. recovery validation → 12. diagnostics dashboard.

26. Final Data, Memory & Operations Statement

NEXUS AI's data architecture is designed around a simple rule: authoritative user data must be durable and recoverable, while derived AI data must be traceable and rebuildable. Conversations, projects, memory and configuration remain structured and user-controlled. RAG indexes, caches and model runtime artifacts are treated as derived or operational data. Observability provides enough information to operate the local AI system without turning telemetry into an uncontrolled copy of private content.

27. Next Engineering Dependency

N55–N62 provide the persistence foundation required by the AI, RAG, agent and UI layers. The next implementation phase can define concrete API contracts, frontend/backend modules, runtime packaging, installation and developer workflows.

Document: N55–N62 Data, Memory & Operations Foundation • **Version:** 0.1 • **Status:** Ready for implementation-level database and operations design